Architecture Note

AI-Powered Eyewear Order Management System

github.com/Hs1000/ai-manufacturer

System Architecture

A two-tier web application. A **React 19 SPA** communicates with a **FastAPI backend** over REST/JSON. Persistence is handled by **SQLite via SQLAlchemy**. Two offline-trained scikit-learn models are embedded directly in the backend process — no external AI API calls are made at inference time.

Request Flow

```
Browser (React 19 + Vite) → HTTPS/JSON → FastAPI (Render) → SQLite + .pkl models
```

Deployment

Layer	Technology	Host	URL
Frontend	React 19 + Vite	Vercel	via Vercel (auto-assigned)
Backend API	FastAPI + uvicorn	Render (free)	https://ai-manufacturer.onrender.com
Database	SQLite (eyewear.db)	Render disk (ephemeral)	co-located with backend
ML Models	.pkl files via joblib	Render disk (from git)	loaded at process start

AI Models

1. SLA Breach Classifier — RandomForestClassifier

What it does: Given an in-flight order, outputs a probability (0–1) that the order will breach its SLA. Used by the AI Predictions dashboard and the Alert engine.

Feature	Importance	Why included
elapsed_hours	59.4%	Primary signal — time consumed vs. SLA target
lens_type	37.9%	SLA target varies per type (48h / 72h / 120h)
status	2.7%	Current production stage affects remaining risk

Why Random Forest over alternatives: The breach signal is nonlinear — probability jumps sharply near the SLA deadline, not gradually. Random Forest captures this without manual feature engineering. The dataset (~500 rows) is too small for a neural network (would overfit) and too nonlinear for logistic regression. **predict_proba()** gives calibrated probabilities, enabling HIGH / MEDIUM / LOW risk bucketing (≥80% / 40–79% / <40%) without retraining when thresholds change.

2. Inventory Demand Forecast — RandomForestRegressor

What it does: Given a lens SKU (type + power + index + coating), predicts units consumed. Output drives the recommended_stock field (predicted × 1.30 safety buffer) on the Inventory Health dashboard.

Feature	Type	Values
lens_type	Categorical (encoded)	Single Vision, Progressive, Blue Light
power	Categorical (encoded)	-1.00 to -3.00
lens_index	Categorical (encoded)	1.50, 1.60, 1.67

coating	Categorical (encoded)	Anti Glare, Blue Light, UV Protection

Why Random Forest over time-series models: Demand per SKU is predicted as a category aggregate, not as a time series. The dataset is structured as transaction totals per SKU (from InventoryTransaction), not timestamped sequences — so ARIMA / Prophet are the wrong shape. Random Forest handles multi-categorical feature interactions (type × power × coating) natively and requires no hyperparameter tuning at this scale.

SLA Rules Engine — Hardcoded Thresholds

SLA targets are defined in **sla.py** and used to label training data and compute hours_remaining at runtime. The breach classifier is trained against these thresholds.

Lens Type	SLA Target	Used for
Single Vision	48 hours	Training label + runtime SLA countdown
Progressive	120 hours	Training label + runtime SLA countdown
Blue Light	72 hours	Training label + runtime SLA countdown

Why No External AI API?

The application intentionally avoids external LLM or ML APIs (OpenAI, Anthropic, Vertex AI) for core prediction features:

Concern	Detail
Latency	Embedded joblib inference is sub-millisecond. An API call per order on a 500-row list would block renders.
Cost	Per-order API calls at page-load scale would be unpredictable and expensive.
Data privacy	Order data, power prescriptions, and identifiers stay on-premise. No PII leaves the server.
Determinism	Embedded .pkl models produce identical output for the same input across deploys.

Note: An LLM integration is listed as a future enhancement for natural-language operational insights — summarisation over aggregates is a better fit for an API call than per-order classification.

Alert Engine — Rule-Based (alerts.py)

Alert Type	Trigger Condition	Severity
LOW_STOCK	LensInventory.quantity < 30	MEDIUM
HIGH_RISK_ORDER	Order.status == 'QC'	HIGH
DEMAND_SPIKE	InventoryForecast.predicted_demand > 10	INFO